you'll learn how to create generalized versions of filters that work with *multiple* endpoint handlers.

The `fruit` API we've been working with in this chapter contains several endpoint handlers that take multiple parameters. The `MapPost` handler, for example, takes a `string id` parameter and a `Fruit fruit` parameter:

```
app.MapPost("/fruit/{id}", (string id, Fruit fruit) => { /* */ });
```

In this example, the `id` parameter is listed first, but there's no requirement for that to be the case. The parameters to the handler could be reversed, and the endpoint would be functionally identical:

```
app.MapPost("/fruit/{id}", (Fruit fruit, string id) => { /* */ });
```

Unfortunately, with this order, the `ValidateId` filter described in listing 5.9 won't work. The `ValidateId` filter assumes that the first parameter to the handler is `id`, which isn't the case in our revised `MapPost` implementation.

ASP.NET Core provides a solution that uses a factory pattern for filters. You can register a filter factory by using the `AddEndpointFilterFactory()` method. A *filter factory* is a method that returns a *filter function.* ASP.NET Core executes the filter factory when it's building your app and incorporates the returned filter into the filter pipeline for the app, as shown in figure 5.10. You can use the same filter-factory function to emit a different filter for each endpoint, with each filter tailored to the endpoint's parameters.